МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ федерального государственного бюджетного образовательного учреждения высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»
Техникум Пермского института (филиала)

Индивидуальное задание

по дисциплине «Технология разработки программного обеспечения»

Тема: «Разработка модуля импорта товаров в электронный магазин из CSV-файлов с хранением данных в коллекциях в памяти.»

Студента группы ИПо-42
Южанинова Арина Юрьевна

Вариант - 1

2026 г.

Содержание

Введение 3
Глава 1. Описание проекта и используемые технологии 5
1.1. Назначение приложения 5
1.2. Используемые технологии 5
1.3. Среда разработки 5
Глава 2. Архитектура проекта 6
2.1. Общая структура проекта 6
2.2. Использование паттерна MVC 6
2.3. Организация бизнес-логики 7
3.1. Модель товара 8
3.2. Модель результата импорта 8
3.3. View-модели для передачи данных 8
Глава 4. Реализация функционала приложения 10
4.1. Чтение и разбор CSV 10
4.2. Валидация данных и обработка ошибок 10
4.3. Работа со списком товаров 10
Глава 5. Пользовательский интерфейс 11
5.1. Навигация и страница импорта 11
5.2. Страница списка товаров 12
Глава 6. Процесс разработки и тестирование 13
6.1. Этапы разработки 13
6.2. Возникшие сложности 13
6.3. Тестирование приложения 13
Заключение 15

Введение

В современном мире информационные технологии играют ключевую роль в автоматизации бизнес-процессов. Электронная коммерция и складской учет требуют оперативной обработки больших объемов данных о товарах. Ручной ввод номенклатуры из бумажных носителей или таблиц Excel является трудоемким процессом, который занимает много времени и подвержен риску возникновения ошибок (опечатки в артикулах, неверное указание цен или единиц измерения). В связи с этим особую актуальность приобретает разработка программных модулей, способных автоматизировать процесс загрузки и первичной валидации данных.

Тема проекта и основная концепция были предложены преподавателем в качестве практического задания по разработке программного модуля с использованием современных веб-технологий.

Целью данной работы является разработка веб-приложения на платформе ASP.NET Core MVC для импорта товаров из CSV-файлов, их проверки на корректность и сохранения в оперативной памяти.

Для достижения поставленной цели были сформулированы следующие задачи:

Изучить формат CSV-файлов и реализовать алгоритм их чтения.

Спроектировать модель данных (класс Product), включающую основные поля и дополнительные параметры (бренд, гарантия, модель).

Разработать сервис валидации, проверяющий обязательные поля, числовые значения и уникальность артикулов.

Реализовать веб-интерфейс (представления) для загрузки файлов и просмотра списка товаров.

Написать модульные тесты для проверки работоспособности логики импорта.

В ходе выполнения работы были реализованы следующие функции: чтение CSV-файлов, валидация полей товара с выводом списка ошибок,

обработка дубликатов артикулов, а также просмотр списка успешно загруженных товаров на отдельной странице.

Практическая значимость работы заключается в том, что разработанный модуль позволяет сократить время на рутинный ввод данных и исключить человеческий фактор при загрузке номенклатуры, что особенно важно для организаций с большим товарооборотом.

Приложение было разработано на языке программирования C# с использованием фреймворка ASP.NET Core MVC. Для разделения интерфейса и бизнес-логики использовался паттерн MVC. Тестирование осуществлялось с помощью библиотеки xUnit. Разработка велась в среде Visual Studio Code на операционной системе macOS.

Глава 1. Описание проекта и используемые технологии

1.1. Назначение приложения

Разработанное приложение представляет собой веб-модуль для импорта товаров. Программа позволяет загружать данные из файлов формата CSV (как по указанному пути, так и через форму выбора файла с компьютера), проверять их на соответствие требованиям и сохранять корректные записи в коллекцию в памяти.

Данный модуль может использоваться как вспомогательный инструмент для администраторов интернет-магазинов или сотрудников склада, которым необходимо быстро обновить базу данных товаров на основе файла, полученного от поставщика.

Приложение позволяет:

— валидировать обязательные поля (артикул, название, цена, остаток);

— проверять дополнительные поля по варианту «Электроника» (бренд, гарантия от 0 до 60 месяцев);

— выводить ошибки с указанием конкретной строки файла;

— просматривать список загруженных товаров.

1.2. Используемые технологии

Для реализации проекта был выбран современный технологический стек:

Технология	**Назначение**
C#	Основной язык программирования
ASP.NET Core MVC	Создание веб-приложения и маршрутизация
HTML/CSS (Razor)	Разметка интерфейса (страницы)
.NET 8	Платформа приложения
xUnit	Модульное тестирование

1.3. Среда разработки

Разработка велась в среде Visual Studio Code на операционной системе macOS. Для запуска проекта использовался терминал с командами .NET CLI. Проверка работы программы осуществлялась через веб-браузер.

Глава 2. Архитектура проекта

2.1. Общая структура проекта

Проект разделен на несколько логических частей, что соответствует паттерну MVC и обеспечивает разделение ответственности:

— Controllers (контроллеры, обрабатывающие запросы);

— Models (модели данных);

— Services (бизнес-логика);

— Views (HTML-шаблоны);

— wwwroot (статические файлы, включая папку csv);

— ECommerceCsvImport.Tests (проект с тестами).

Такое разделение позволило сделать проект более понятным, упростило навигацию по коду и облегчило его дальнейшую поддержку.

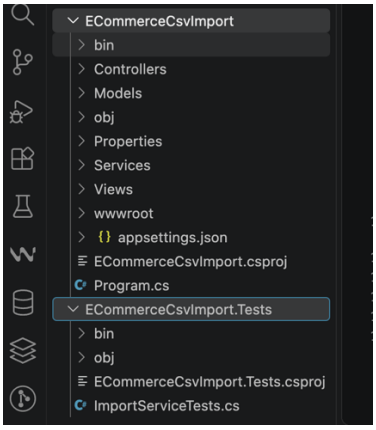


Рисунок 1 – Структура папок проекта в Visual Studio Code

2.2. Использование паттерна MVC

Паттерн MVC (Model-View-Controller) разделяет приложение на три взаимодействующие части:

— Model отвечает за хранение данных (класс Product);

— View представляет пользовательский интерфейс (страницы .cshtml);

— Controller связывает модель и представление, обрабатывая действия пользователя.

Благодаря MVC, контроллер получился «тонким»: он не содержит бизнес-логику, а лишь перенаправляет запросы в сервисы. Это позволяет легко тестировать логику импорта отдельно от веб-интерфейса.

2.3. Организация бизнес-логики

Для работы с данными созданы отдельные сервисы:

— InMemoryProductRepository (хранилище товаров в памяти);

— ProductImportService (логика чтения и проверки CSV);

— CsvParser (разбор строк с учетом кавычек).

Глава 3. Модель данных

3.1. Модель товара

Основная модель проекта — класс Product. Она содержит поля: артикул (Sku), название, описание, цену, валюту, категорию, остаток, флаг активности (IsActive), а также дополнительные поля варианта «Электроника»: бренд (Brand), гарантия (WarrantyMonths) и модель (ModelNumber).

Использование строго типизированных свойств (например, decimal для цены и int для количества) позволяет избежать ошибок при математических операциях и гарантирует, что в базу не попадут некорректные типы данных.

3.2. Модель результата импорта

Класс ImportResult используется для сбора статистики. Он содержит:

— общее количество обработанных строк (TotalRows);

— количество успешно импортированных товаров (ImportedCount);

— список ошибок (Errors), где каждая ошибка имеет номер строки и текст сообщения.

Благодаря этой модели, сервис импорта не просто возвращает true/false, а передает контроллеру полную информацию о том, что произошло во время обработки файла, что позволяет вывести подробный отчет пользователю.

3.3. View-модели для передачи данных

Паттерн MVC подразумевает, что в представления (HTML-страницы) не рекомендуется передавать напрямую бизнес-модели. Для этого используются специальные View-модели, которые адаптированы под нужды конкретной страницы.

В проекте созданы две View-модели:

ProductsIndexViewModel — используется на странице списка товаров. Она содержит список товаров (IReadOnlyList<Product>), а также вычисляемые свойства: TotalCount (общее количество товаров) и TotalValue (общая стоимость остатков, вычисляется через LINQ-запрос Products.Sum(...)).

ImportViewModel — используется на странице импорта. Она содержит строку FilePath (путь к файлу, который ввел пользователь) и

объект ImportResult(результаты импорта). Это позволяет при перезагрузке страницы сохранить введенный путь и показать таблицу с результатами.

Таким образом, модель данных проекта спроектирована с учетом требований паттерна MVC: она разделена на сущности для хранения данных, сущности для передачи результатов и View-модели для отображения информации на сайте.

Глава 4. Реализация функционала приложения

4.1. Чтение и разбор CSV

Класс ProductImportService открывает файл и читает его построчно. Для корректного разбора строк (если в названии товара есть запятая) используется парсер CsvParser, который учитывает кавычки.



Рисунок 2 – Фрагмент кода сервиса ProductImportService

4.2. Валидация данных и обработка ошибок

При обработке каждой строки программа проверяет: не пустой ли артикул и название; является ли цена числом больше 0; не отрицательный ли остаток; корректно ли значение is_active; заполнен ли бренд и находится ли гарантия в диапазоне от 0 до 60 месяцев; нет ли дубликата артикула.

Если есть ошибки, товар не добавляется в память, а в общий список записываются сообщения с указанием номера строки.

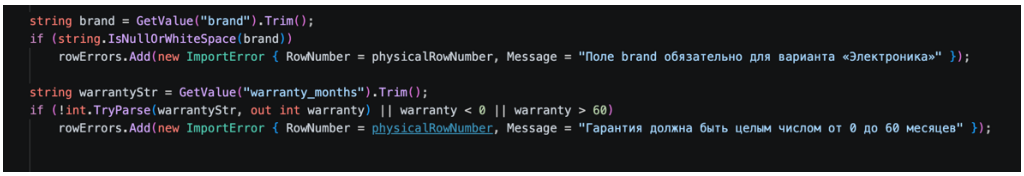


Рисунок 3 – Фрагмент кода с проверкой обязательных полей

4.3. Работа со списком товаров

После успешного импорта данные сохраняются в InMemoryProductRepository. На странице /Products пользователь видит таблицу со всеми загруженными товарами.

Глава 5. Пользовательский интерфейс

5.1. Навигация и страница импорта

Для удобства пользователя в верхней части сайта реализовано навигационное меню («Список товаров» и «Импорт товаров»). Пользователю не нужно вводить адреса страниц вручную.

На странице импорта (/Products/Import) реализовано два способа загрузки данных:

1. Ввод пути к файлу вручную (для тестирования).
2. Выбор файла через стандартное диалоговое окно Windows/Mac (через IFormFile).

После нажатия кнопки импорта страница перезагружается и выводит таблицу с результатами (сколько строк обработано, сколько успешно, сколько ошибок) и списком ошибок, если они есть.

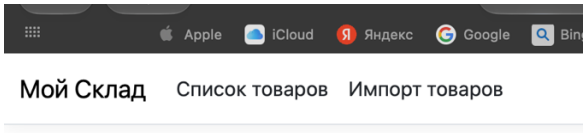


Рисунок 4 – Главная страница сайта с верхним навигационным меню

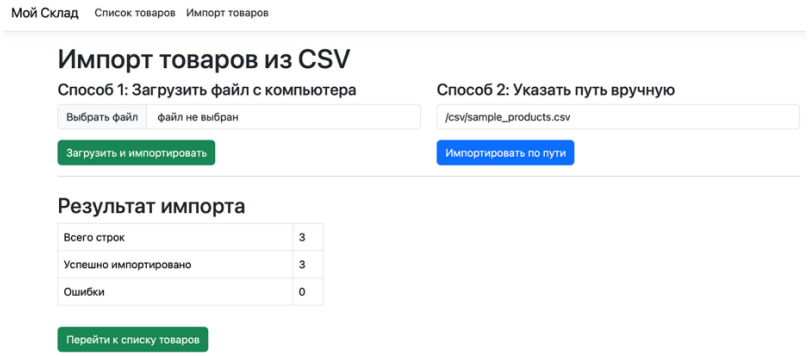


Рисунок 5 – Результат успешного импорта товаров

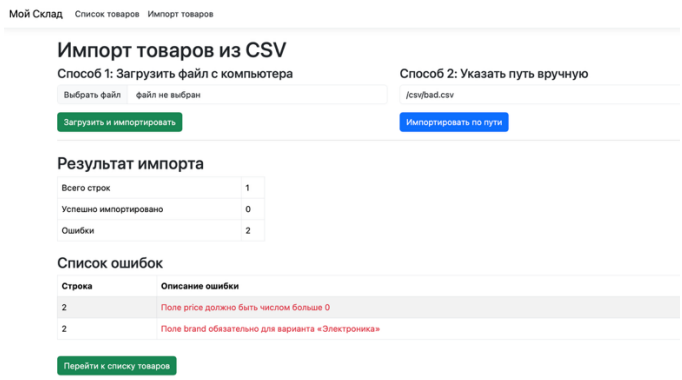


Рисунок 6 – Вывод ошибок при импорте некорректного файла

5.2. Страница списка товаров

После импорта пользователь переходит на страницу /Products. Там отображается сводка (общее количество и суммарная стоимость) и таблица со всеми полями товаров. Таблица имеет адаптивную верстку (Bootstrap), поэтому удобно отображается как на компьютере, так и на экранах поменьше.

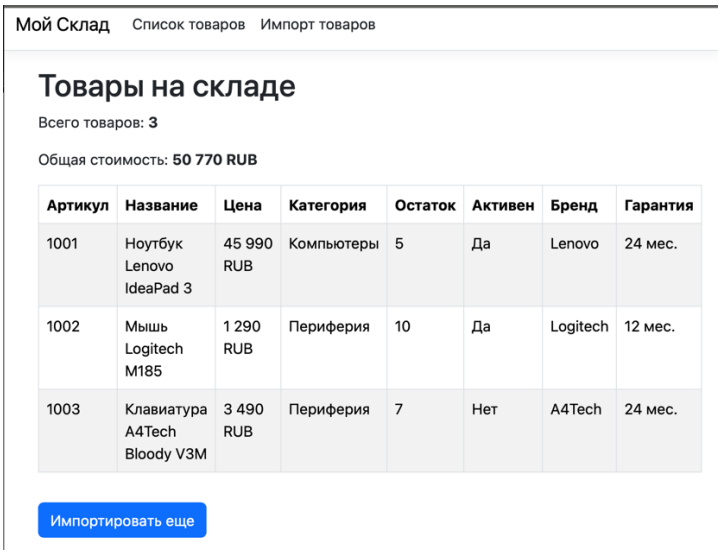


Рисунок 7 – Страница со списком загруженных товаров

Глава 6. Процесс разработки и тестирование

6.1. Этапы разработки

Разработка проекта выполнялась поэтапно:

1. Создание структуры проекта и настройка зависимостей в Program.cs.
2. Написание моделей данных (Product, ImportResult).
3. Реализация парсера CSV и сервиса импорта.
4. Создание репозитория для хранения данных в памяти.
5. Написание контроллера ProductsController и HTML-страниц.
6. Написание модульных тестов.

6.2. Возникшие сложности

Во время разработки возникли сложности с настройкой путей к файлам на macOS (так как проект создавался и запускался на этой ОС). Кроме того, были сложности с подключением тестового проекта (ECommerceCsvImport.Tests) к основному проекту: сборка выдавала ошибки из-за конфликта версий библиотек. Проблема была решена путем корректной настройки ссылок (Reference) между проектами и обновления пакетов через терминал.

6.3. Тестирование приложения

Для проверки работоспособности модуля использовалось два вида тестирования:

1. Ручное тестирование: загрузка различных файлов через браузер. Проверялись сценарии успешного импорта (3 корректные строки), обработка файла с неверной ценой (-500) и пустым брендом.
2. Автоматическое тестирование: были написаны Unit-тесты с использованием фреймворка xUnit. Тесты проверяют успешный импорт, корректную обработку неверной цены и обнаружение дубликатов артикулов. Все тесты успешно пройдены (Passed).

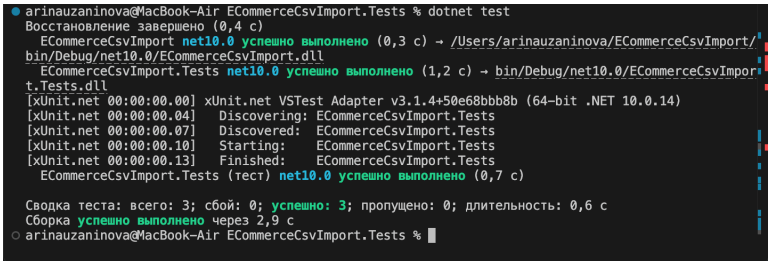


Рисунок 8 – Результат выполнения модульных тестов

Заключение

В ходе выполнения данной работы было разработано веб-приложение для импорта товаров из CSV-файлов на платформе ASP.NET Core MVC. Основной целью проекта являлось создание удобной системы, позволяющей автоматизировать загрузку номенклатуры и исключить ошибки ручного ввода.

В процессе работы были успешно решены все поставленные задачи. Был реализован алгоритм чтения CSV-файлов (с учетом кавычек и запятых), спроектирована модель данных Product с дополнительными полями варианта «Электроника», разработан сервис валидации (проверка цен, остатков и брендов) и создан удобный веб-интерфейс для загрузки файлов и просмотра списка товаров.

В ходе работы я получила ценные практические навыки разработки на языке C#, изучила принципы работы паттерна MVC и внедрения зависимостей (Dependency Injection), а также освоила написание модульных тестов с использованием библиотеки xUnit. Отдельно стоит отметить опыт работы с терминалом и настройки проекта на операционной системе macOS.

Разработанное приложение имеет модульную структуру и может быть доработано в дальнейшем. В перспективе в проект можно добавить подключение реальной базы данных, систему авторизации пользователей, а также функции поиска и фильтрации.

Таким образом, все поставленные задачи были выполнены в полном объеме. В результате было создано работоспособное приложение для импорта товаров, обладающее удобным интерфейсом и надежной системой проверки данных.